

The Interface Is a Variable: Measuring the Cost and Reliability of Purpose-Built UI Representations for LLM Agents

Working draft. Authors: TBD.

Reported numbers are from one model (`gemini-2.5-flash`) on one synthetic store and should be read as a first data point, not as a finished claim about all agents or all applications.

Abstract

Language-model agents often operate software through interfaces built for people: a screenshot (C1), an accessibility tree (C2), or a catalog of tools. We treat the *representation* itself (how the application shows state and available actions) as the thing to vary, not as a fixed cost of automation. Holding the store, the tasks, and a deterministic grader fixed, we compare four views of the same shop: screenshot (C1), accessibility tree (C2), flat tools (C3), and a purpose-built JSON view document (C4) that names the current screen and which actions are allowed on it. We measure success, token cost, steps, and illegal actions (calls the backend rejects).

On this first model, the human surfaces lose. Screenshot (C1) costs about ten times the cheapest condition and completes fewer than a third of tasks. Accessibility tree (C2) is cheaper than a screenshot but noisy. Between the two structured conditions the trade-off is smaller: view document (C4) uses fewer steps, yet costs more tokens than flat tools (C3) and does not improve success. We treat this as a measurement method plus one cell, and we say where a purpose-built agent surface is still expected to help.

1. Introduction

An agent that uses software repeats a short loop. It looks at the application, chooses one action, takes that action, and looks again. Each look is billed as input tokens. The shop does not change across that loop. Only the *form* of the look changes.

Today that form is almost always one built for people. Three inherited channels dominate, and each fails the agent in a different way.

1. **screenshot (C1).** The model sees pixels and must click coordinates. Grounding those clicks is a known failure mode (SeeClick, UI-TARS, Claude computer use). Image cost tracks the size of the viewport, not how much the page actually says, so a blank page and a busy page at the same size cost the same.

2. **accessibility tree (C2).** The model reads a text tree built for assistive technology. The tree is often huge (Mind2Web filters it; UIFormer finds that UI text is 80 to 99 percent of agent token cost). It also names controls the backend will reject, so named clicks become illegal actions.
3. **flat tools (C3).** The model gets function schemas and a short status object, with no description of the current screen. This is close to how MCP exposes capabilities. It is cheap per call once the schema is paid, but the model must remember which view it is on and which arguments are legal.

None of these three was designed to answer, cheaply and clearly, *what is true now and what may be done next*.

view document (C4) is the alternative we measure. It is a JSON object authored from backend state: the current view, the entities on it, and each action with an `enabled` flag and an argument schema. It is not a better screenshot. It is not a pruned DOM. It is a projection of the same backend the grader already trusts.

The product question (should applications ship this the way they ship an API?) is motivation, not the contribution. The contribution is a measurement: hold the store, the tasks, and the grader fixed, vary only the representation, and read a cost-reliability frontier. A representation sits on that frontier only if nothing else is both cheaper and at least as reliable.

What we hold still, and what we change. One shop (MiniShop), ten frozen tasks, one deterministic grader over backend orders, one model at a time, temperature zero, a cap of twenty steps. History is the task, prior actions, and the current observation. The only thing that changes is how the agent sees and acts: screenshot (C1), accessibility tree (C2), flat tools (C3), or view document (C4).

What we found on this cell. Human surfaces are dominated. Screenshot (C1) never completes a success task and costs on the order of 46k input tokens. Accessibility tree (C2) succeeds more often but mixes illegal calls and malformed replies. Flat tools (C3) is perfect here. View document (C4) nearly matches it, uses fewer steps, costs more tokens, and fails one task because the model ignored `enabled: false`. Hypothesis H2 (that view document (C4) dominates) is not supported on this model and this shop. Whether that ordering holds on other models is untested.

Contributions.

1. A controlled comparison at *matched action grain*: flat tools (C3) and view document (C4) expose the same operations, so a win for view document (C4) cannot be "fewer buttons."
2. A cost-reliability reading with paired per-task intervals, rather than a success-only leaderboard.
3. First evidence on one shop and one model, including a model-free observation-cost baseline and an execution-based grader.

Questions.

- **RQ1.** Does the representation change tokens, steps, success, and illegal actions on a fixed task set?
- **RQ2.** Is some representation cheaper and at least as reliable as reusing the human page? In particular, does view document (C4) dominate screenshot (C1) and accessibility tree (C2)?
- **RQ3.** Is the ordering stable across general models?

The matching hypotheses are H1 (cost and reliability differ), H2 (view document (C4) is on the frontier and screenshot (C1) is dominated), and H3 (the *order* of conditions is stable across models even if the *size* of the gap is not). On this first model, H1 holds at a coarse grain (human surfaces versus structured ones). H2 does not hold for view document (C4) versus flat tools (C3). H3 is untested.

How to read the rest. Section 2 defines terms. Section 3 places the work. Section 4 describes MiniShop and the four conditions, with a real view document (C4) example. Section 5 defines metrics. Section 6 reports results. Section 7 says what this cell shows and what later cells must show. Section 8 sketches how an application could ship a view document. Section 9 lists limits.

2. Important terms

We use the same labels everywhere: method name, then the condition id.

- **screenshot (C1).** A picture of the human page. The agent clicks, types, and scrolls.
- **accessibility tree (C2).** A text tree of that page (roles and names). The agent clicks or fills named elements.
- **flat tools (C3).** A catalog of functions and a short status object. No current-view document. The model must track state itself.
- **view document (C4).** A JSON object with `view`, `state`, `entities`, and `affordances`. Each affordance has an `enabled` flag and an argument schema.
- **Affordance.** An action the interface exposes (`open_product`, `set_size`, `pay`, and so on).
- **Enabled flag.** Whether that action is valid in the current state.
- **Matched action grain.** Flat tools (C3) and view document (C4) offer the same operations. Differences between them come from the document, not from a different button set.
- **Illegal action.** The backend rejects the call (HTTP 400).
- **Malformed action.** The model reply is not a usable action (wrong shape). Tracked separately from illegal actions.
- **Ignored-affordance rate.** How often the model tries an action marked unavailable. Task t10 is the running example.
- **Step.** One observe, decide, act cycle. The cap is 20.
- **Execution-based grader.** A checker over backend orders. Never another model.

- **Cost-reliability frontier.** Token cost versus success. A point is on the frontier if nothing is both cheaper and at least as reliable.
- **Refusal task.** The correct behavior is to decline. Reported separately, because doing nothing can pass.

3. Related work

Most prior work asks which *agent* is better at using a computer, and treats the interface as a property of the benchmark. We invert that: the model is fixed and the interface is the independent variable.

Reusing the human page. DOM and accessibility agents (Mind2Web, WebArena) are accessibility tree (C2). Screenshot and pixel systems (Claude computer use, UI-TARS, SeeClick) are screenshot (C1). Grounding is hard, and screenshots often cost on the order of 1k to 2k tokens each. Our question is not how to ground pixels better. It is whether a non-pixel, agent-authored surface is better for ordinary general models. Specialized pixel-native models are left for a later check.

Compressing the human tree. UIFormer and Prune4Web prune the human DOM or accessibility tree. UIFormer reports that UI text is 80 to 99 percent of agent token cost, with about 49 to 56 percent reduction. That line improves accessibility tree (C2). View document (C4) is authored from backend state, not derived from the human page, and we measure it as a controlled arm, not as a plug-in optimizer.

Agent-facing protocols. A2UI is the opposite direction: an agent draws UI for a person. MCP is a capability catalog, close to flat tools (C3). View document (C4) is application to agent, per view, and constraint-carrying. We use it as a measurement probe, not as a proposed standard.

Benchmarks. WebArena, VisualWebArena, Mind2Web, WebShop, AndroidWorld, and MiniWoB++ vary the agent, fix the interface, and grade by execution. We reuse execution grading. We do not reuse the goal. We also report a model-free observation-cost baseline, which those benchmarks do not.

Table R1. How this study differs (verified sources only).

Strand	Representative work	What it does	How this study differs
DOM / accessibility agents	Mind2Web; WebArena	Operate the human page from HTML or a tree	That is accessibility tree (C2), one of four conditions
Screenshot / pixel computer-use	Claude computer use; UI-TARS; SeeClick	See pixels, emit coordinates	That is screenshot (C1). Representation, not better grounding
Optimizing the human tree	UIFormer; Prune4Web	Compress a human-derived tree	Closest neighbor. View document (C4) is authored from backend state
Agent-UI protocols	A2UI; MCP	A2UI is agent to human. MCP is a tool catalog	View document (C4) is application to agent, matched to flat tools (C3)
GUI / web benchmarks	WebArena; VisualWebArena; Mind2Web; WebShop; AndroidWorld; MiniWoB++	Vary the agent, fix the interface	We fix the agent and vary the interface

What this is not. "Representation matters" is already known (Mind2Web filters the DOM; SeeClick is motivated by lengthy HTML; UIFormer measures UI token share). Execution grading is not new. Token-efficient UI trees are not new. Agent-facing surfaces are already discussed (A2UI, MCP). Constrained action masking is treated as a known idea; we do not attach an unchecked citation. The claim here is the controlled A/B/C/D, not those facts. Scope (one shop, ten tasks, one model, a hand-authored view document (C4)) further bounds novelty.

4. Study design

The shop. MiniShop is a small catalogue with product pages, a cart, and checkout. Backend state is the single source of truth for grading. The human HTML includes promo copy, search, decorative filters, and help links, so screenshot (C1) and accessibility tree (C2) see a page built for people, not a stripped agent shell. Sold-out sizes are visible but disabled on that page, so a screenshot or tree agent can still try them. Structured conditions act through a JSON API. Screenshot (C1) and accessibility tree (C2) drive the human pages in a browser and do not use that JSON API to act.

Why only the interface changes. If we also changed the shop, the tasks, the grader, or the model, we could not blame the interface. We hold those fixed and vary only how the agent sees and acts.

Table I1. Held fixed versus varied.

Held fixed	Varied
MiniShop, one backend, one grader	How the agent <i>sees</i> the store
Ten frozen tasks	How the agent <i>acts</i>
One model per comparison; temperature 0; step cap 20	Presence of view document (C4) versus flat tools (C3)
History: task, prior actions, current observation	

Screenshot (C1) and accessibility tree (C2) cannot share the same action vocabulary as the structured conditions (coordinates and named elements versus function calls). That is a real confound when comparing human surfaces to structured ones, and we state it as such. The comparison that isolates the *document* is flat tools (C3) versus view document (C4).

Matched action grain. Both structured conditions expose the same seven operations:

`open_product`, `set_size`, `add_to_cart`, `go_catalog`, `go_checkout`, `set_address`, `pay`. Flat tools (C3) omits the view document. View document (C4) adds current-view identity, `enabled` flags, and argument schemas. If view document (C4) had fewer or easier actions, a win could be "fewer buttons." Matched grain blocks that. What it does not yet separate is compactness versus constraint-carrying. Those stay confounded until the ablation in Section 6.5. Models are never mixed across conditions in one comparison.

The four conditions.

Condition	What the agent sees	How it acts
screenshot (C1)	Rendered image of the human page	Click coordinates, type, scroll
accessibility tree (C2)	Text tree of that page	Click or fill named elements
flat tools (C3)	Function catalog; no current view	Call those functions
view document (C4)	JSON: view, state, entities, affordances	Invoke an enabled affordance with typed arguments

Tasks and grader. Ten frozen tasks (seven expected to succeed, three expected to be refused). The grader checks the backend order record. It is never another model. Success tasks require a matching product, size, and shipping address. Refusal tasks require that a forbidden order is *not* placed: t03 (Harbor Blue Tee size M, sold out), t05 (pay on an empty cart), t07 (Signal Cap size S, sold out). A do-nothing agent can pass a refusal by inaction, so refusals are reported separately and must not inflate the headline success rate.

Table T1. Frozen MiniShop tasks.

ID	Expect	Instruction (abridged)
t01	success	Navy Crew Tee, size M, to 18 Cedar Ave, Portland
t02	success	Harbor Blue Tee, size L, to 18 Cedar Ave, Portland
t03	refusal	Harbor Blue Tee, size M (sold out)
t04	success	Studio Mug, to 9 Pine Street, Austin
t05	refusal	Pay with an empty cart
t06	success	Ash Hoodie, size S, to 4 Market Road, Seattle
t07	refusal	Signal Cap, size S (sold out)
t08	success	Field Notebook, to 100 King St, Boston
t09	success	Day Bottle and Crew Socks, to 18 Cedar Ave, Portland
t10	success	Navy Crew Tee, size L, to 77 Oak Lane, Denver

How a run works. One run is one (condition, task, repeat). The agent loops: observe, emit one action, apply it, observe again, until success or the step cap. A full sweep is 4 conditions $\times$ 10 tasks $\times$ 5 repeats (200 runs). A deterministic oracle that replays the correct policy is run first on flat tools (C3) and view document (C4). It reaches 100% on both, so a later model miss is the model, not the harness. Each run is a JSONL trace with per-step usage and a final rollup. Conditions are launched separately so a crash in one cannot abort the others.

4.1 An example of view document (C4)

The document always has four keys. `view` is which screen the agent is on. `state` is the session fields the view depends on. `entities` are the objects shown. `affordances` are the actions, each with `enabled` and an `input` schema.

The listing below is a real document from MiniShop, not a sketch. The agent has opened Navy Crew Tee, chosen size L, and added it to the cart. It is still on the product page. It has not gone to checkout. Setting a shipping address is therefore illegal. The document says so: `set_address` and `pay` are `enabled: false`. `go_checkout` is enabled. The product-id enum is shortened in print; the live document lists every catalog id.

```
{
  "view": "product",
  "state": {
    "product_id": "tee-navy",
    "selected_size": "L",
    "cart_count": 1,
    "address": "",
    "order_count": 0
  },
  "entities": {
    "product": {
      "id": "tee-navy",
      "name": "Navy Crew Tee",
      "price": 28,
      "sizes_in_stock": ["S", "M", "L"]
    },
    "cart": [
      {
        "product_id": "tee-navy",
        "name": "Navy Crew Tee",
        "size": "L",
        "price": 28
      }
    ]
  },
  "affordances": [
    {
      "id": "open_product",
      "enabled": true,
      "input": {
        "type": "object",
        "properties": {
          "product_id": {
            "type": "string"
          }
        },
        "required": [
          "product_id"
        ]
      }
    },
    {
      "id": "set_size",
      "enabled": true,
      "input": {
        "type": "object",
        "properties": {
          "size": {
            "type": "string",
            "enum": [
              "S",
              "M",
              "L"
            ]
          }
        },
        "required": [
          "size"
        ]
      }
    },
    {
      "id": "add_to_cart",
      "enabled": true,
      "input": {
        "type": "object",
        "properties": {
          "product_id": {
            "type": "string"
          },
          "size": {
            "type": "string",
            "enum": [
              "S",
              "M",
              "L"
            ]
          }
        },
        "required": [
          "product_id",
          "size"
        ]
      }
    },
    {
      "id": "go_catalog",
      "enabled": true,
      "input": {
        "type": "object",
        "properties": {
          "product_id": {
            "type": "string"
          }
        },
        "required": [
          "product_id"
        ]
      }
    },
    {
      "id": "go_checkout",
      "enabled": true,
      "input": {
        "type": "object",
        "properties": {
          "product_id": {
            "type": "string"
          },
          "size": {
            "type": "string",
            "enum": [
              "S",
              "M",
              "L"
            ]
          }
        },
        "required": [
          "product_id",
          "size"
        ]
      }
    },
    {
      "id": "set_address",
      "enabled": false,
      "input": {
        "type": "object",
        "properties": {
          "address": {
            "type": "string"
          }
        },
        "required": [
          "address"
        ]
      }
    },
    {
      "id": "pay",
      "enabled": false,
      "input": {
        "type": "object",
        "properties": {
          "product_id": {
            "type": "string"
          },
          "size": {
            "type": "string",
            "enum": [
              "S",
              "M",
              "L"
            ]
          },
          "address": {
            "type": "string"
          }
        },
        "required": [
          "product_id",
          "size",
          "address"
        ]
      }
    }
  ]
}
```

This is the t10 failure, in one object. The valid next step is `go_checkout`. The model instead called `set_address` while `enabled` was false. The backend returned HTTP 400. Surface and backend agree; a regression test pins that agreement. The miss is the model, not a lying document.

The document is a hand-written function of backend state, so it is an *upper bound* on how good an agent surface can be: it is faithful by construction. Enable rules match the human UI:

`add_to_cart` only when a valid size is selected on a product view; `set_address` only on checkout; `pay` only on checkout with a non-empty cart and address; `set_size` enum exactly the in-stock sizes. Tests forbid the document from advertising an action the backend would reject.

5. What we measure

Per run. Success (binary; for refusals, correctly declining). Input and output tokens summed over steps. Step count. Illegal actions. Malformed actions, counted separately so formatting failures are not absorbed into the illegal rate.

Why tokens split this way. Total input is roughly (fixed overhead per step) plus (current observation). A representation can win by being lighter per look, or by needing fewer looks.

Text conditions (accessibility tree (C2), flat tools (C3), view document (C4)). Cost is the length of the serialized observation under a fixed tokenizer (`o200k_base` for the deterministic baseline). Flat tools (C3) also re-sends the full tool schema on every call. View document (C4) pays for the document. Accessibility tree (C2) pays for the human tree.

Median per-step input along the canonical path:

Condition	system	task+prior	tool schema	observation	per-step total
screenshot (C1), estimate	186	62	n/a	1105 (image)	1353
accessibility tree (C2), measured	188	62	n/a	~650 (tree)	~900
flat tools (C3)	159	62	490	25	736
view document (C4)	188	62	n/a	422	672

The flat tools (C3) schema (490 tokens) dwarfs its 25-token status object. View document (C4)'s whole extra cost is the document (422). Accessibility tree (C2) is measured from live browser traces. The other three are reconstructed with no model call.

screenshot (C1), tiling, not tokenization. A screenshot is billed by resolution tiles, not by how busy the page looks. A blank page and a busy page at the same viewport cost the same. On a 1280×800

viewport the gpt-4o-style high-detail estimate is about 1105 image tokens (6 tiles). Providers differ: OpenAI itemizes image tokens; Vertex/Gemini fold them into `prompt_tokens`. That is why our `gemini-2.5-flash` screenshot (C1) runs show `image_tokens = 0` and about 46k input tokens per task. The image is billed. It is just not itemized. We therefore read screenshot (C1) cost from total input tokens. Tiling constants are provider-specific; the gpt-4o numbers are a labeled estimate, not a universal formula.

Analysis. We report a per-(model, condition) table and a cost-reliability frontier (median input tokens versus success). Because the same tasks run under every condition, we also report paired per-task differences between view document (C4) and flat tools (C3), with bootstrap 95% confidence intervals resampled over tasks, success and refusal separately. We never pool models.

Latency is planned and not yet instrumented. Time-to-completion weights round trips differently from tokens, so it could move the structured comparison without changing Table 1's token ranks.

6. Results

One model, one shop, ten tasks. Read coarse orderings as the robust part. Treat the fine gap between flat tools (C3) and view document (C4) as provisional.

6.1 Model-free observation cost

Before any model runs, we replay each task's canonical path and count observation tokens. Median per-step observation cost is about 1105 for screenshot (C1), 512 for flat tools (C3) (schema re-sent each step), and 453 for view document (C4). Text is about 2.4 times cheaper per look than a screenshot. This is the observation term only, on the shortest path. It does not include step-count effects.

6.2 Oracle control

A deterministic agent that follows the correct policy reaches 100% on flat tools (C3) and view document (C4). Oracle tokens are locally tokenized, not provider-billed, and the flat tools (C3) tool schema is omitted from that count because the oracle loop sends schemas out of band. Screenshot (C1) and accessibility tree (C2) are out of scope for the oracle (they would need pixel or element oracles).

6.3 First model run: `gemini-2.5-flash` (N = 5)

One general model, temperature 0, step cap 20, all four conditions, all ten tasks, five repeats (200 runs). Table 1 is the aggregate. Human pages for screenshot (C1) and accessibility tree (C2) are the catalogue, a sold-out product, checkout, and confirmation. Interval estimates are percentile bootstrap 95% CIs over the 50 task-repeat units per cell.

Table 1. Per-condition results, `gemini-2.5-flash`, N=5.

Condition	Success (95% CI)	Median input tokens (95% CI)	Median steps	Illegal/step	Malformed/step
screenshot (C1)	30% [18, 42]	~46,338 [46,324, 46,368]	20	0.000	0.000
accessibility tree (C2)	72% [60, 84]	~6,770 [5,835, 9,714]	7	0.198	0.116
flat tools (C3)	100% [100, 100]	~3,134 [3,118, 3,154]	7	0.000	0.000
view document (C4)	92% [84, 98]	~4,455 [4,413, 4,491]	6	0.016	0.000

Across 200 runs the traces sum to about 2.82M input tokens and 35k output tokens, about \$0.93 at the recorded Flash rates (\$0.30 per 1M input, \$2.50 per 1M output). By condition: screenshot (C1) 2.11M in / 19.9k out; accessibility tree (C2) 385k / 9.4k; flat tools (C3) 138k / 1.7k; view document (C4) 193k / 4.1k. Screenshot (C1) is about 73% of the bill.

Success. Screenshot (C1) passes 30% [18, 42] and *none* of the seven success tasks (its passes are refusals). Accessibility tree (C2) reaches 72% [60, 84] but mixes backend rejections and malformed JSON. Flat tools (C3) is 100%. View document (C4) is 92% [84, 98], with the entire gap on t10. Intervals are wide. The structured success gap is one task.

Tokens. Screenshot (C1) costs about 10 times view document (C4) and about 15 times flat tools (C3). Vertex folds image tokens into `prompt_tokens`, so that cost is in total input. Among text conditions, flat tools (C3) is cheapest, view document (C4) next, accessibility tree (C2) most expensive and most variable. The model-run order (flat tools (C3) cheaper than view document (C4)) *reverses* the minimal-path baseline in Section 6.1: once real step counts are in play, the view document (C4) grows as the cart fills.

Steps. View document (C4) uses the fewest median steps (6). Flat tools (C3) uses 7. Screenshot (C1) sits at the cap (20) because it never completes a success task. Accessibility tree (C2) has median 7 and a long tail (it hits the cap on the two-item task t09).

Illegal and malformed. Screenshot (C1) and flat tools (C3) are clean on both (0.000). View document (C4) has 0.016 illegal/step, all on t10. Accessibility tree (C2) is the outlier (0.198 illegal/step, 0.116 malformed/step). Screenshot (C1)'s illegal rate of 0.000 is not virtue: a coordinate click rarely trips the backend flag. Its failure is in success, steps, and tokens.

Table 2. Per-task readout, `gemini-2.5-flash`, N=5. Cell = success rate · median steps · median input tokens. Refusal tasks marked *r*.

Task		screenshot (C1)	accessibility tree (C2)	flat tools (C3)	view document (C4)
t01		0% · 20 · 46,368	20% · 12 · 11,702	100% · 7 · 3,154	100% · 6 · 4,498
t02		0% · 20 · 46,368	100% · 11 · 10,809	100% · 7 · 3,154	100% · 6 · 4,486
t03	<i>r</i>	100% · 2 · 4,184	100% · 2 · 1,997	100% · 2 · 1,081	100% · 2 · 1,597
t04		0% · 20 · 46,268	0% · 6 · 5,835	100% · 7 · 3,118	100% · 6 · 4,423
t05	<i>r</i>	100% · 20 · 46,018	100% · 3 · 2,605	100% · 2 · 713	100% · 2 · 1,528
t06		0% · 20 · 46,328	100% · 10 · 9,714	100% · 7 · 3,144	100% · 6 · 4,491
t07	<i>r</i>	100% · 20 · 46,348	100% · 2 · 1,990	100% · 2 · 1,079	100% · 1 · 905
t08		0% · 20 · 46,324	100% · 7 · 6,690	100% · 7 · 3,124	100% · 6 · 4,413
t09		0% · 20 · 46,428	0% · 20 · 21,658	100% · 12 · 5,833	100% · 10 · 7,867
t10		0% · 20 · 46,368	100% · 7 · 6,770	100% · 7 · 3,154	20% · 6 · 4,544

Screenshot (C1) hits the step cap on every task it does not refuse, and fails all seven success tasks. Its cost is nearly flat (~46k) because image cost is set by viewport. Accessibility tree (C2) fails t01, t04, and t09 (t09 step-capped at ~21.7k tokens). Flat tools (C3) is clean on every task. View document (C4) misses only t10 (20%); elsewhere it matches flat tools (C3) on success, uses one fewer step, and costs more tokens.

Paired view document (C4) versus flat tools (C3). Average each task over five repeats, take view document (C4) minus flat tools (C3), bootstrap a 95% CI over tasks.

Table 3. Paired differences, `gemini-2.5-flash`. Positive means view document (C4) is higher.

Task set	n	Metric	Mean diff (C4 minus C3)	95% CI	Excludes 0?
success	7	success	-0.114	[-0.343, 0.000]	no
success	7	input tokens	+1,404	[+1,317, +1,549]	yes
success	7	steps	-1.17	[-1.51, -1.00]	yes
refusal	3	success	0.000	[0.000, 0.000]	no
refusal	3	input tokens	+386	[-174, +815]	no
refusal	3	steps	-0.33	[-1.00, 0.00]	no

On success tasks, view document (C4) **saves steps** (CI excludes zero) and **costs more tokens** (CI excludes zero; all seven tasks positive). It does **not** significantly change success (CI includes zero; the whole point estimate is t10). Flat tools (C3) is therefore Pareto-preferred here: cheaper and at

least as reliable. H2's claim that view document (C4) dominates is not supported on this cell. On refusals, nothing separates them.

Task t10, plainly. Buy a Navy Crew Tee in size L, ship to 77 Oak Lane, Denver. Valid order: open product, add to cart, go to checkout, *then* set the address. The model called `set_address` while still on the product view. Section 4.1 is that state. The document had `enabled: false`. The backend rejected it. Reproduced on 4 of 5 main-sweep repeats and 5 of 5 on a t10-only re-run, same illegal step. A truthful document does not help a model that does not read `enabled`. That ignored-affordance rate is what a second model must measure.

6.4 Second model (RQ3), not yet run

Whether "flat tools (C3) is Pareto-preferred" generalizes is open. A second general model would test, in particular, whether other models obey `enabled: false` on t10.

6.5 Constraint-pruning ablation, not yet run

To separate compactness from constraint-carrying, strip `enabled` flags and argument enums from view document (C4), keep view and entities, and measure whether the step saving survives.

7. Discussion

7.1 What this cell shows

The cell is `gemini-2.5-flash` × MiniShop, N=5, ten tasks.

- **RQ1, coarse.** Representation moves success, tokens, steps, and error rates. Screenshot (C1) is about 10 times view document (C4) and about 15 times flat tools (C3) in median input tokens, succeeds 30%, and hits the step cap on every success task. Accessibility tree (C2) is 72% with the highest error rates. Both structured conditions clear the human surface.
- **RQ2, human surfaces.** Screenshot (C1) and accessibility tree (C2) are dominated here. That part of H2 holds.
- **RQ2, structured pair.** H2's claim that view document (C4) dominates is *not* supported. Flat tools (C3) is Pareto-preferred on tokens and success. The view document (C4) mechanism (fewer steps, -1.17 per success task) is present and not large enough to offset +1,404 input tokens.
- **t10 is model behavior.** The document was truthful. The model ignored it.

This is the *least favorable* setting for a purpose-built surface: tiny state, short tasks, and a model that ignores flags. Flat tools (C3)'s memory burden is light. View document (C4) still pays to send a growing document.

7.2 What later cells must show

These are predictions, not results.

1. **Second general model, N=5 (RQ3 / H3).** If another model obeys `enabled: false` on t10, view document (C4) success should rise toward flat tools (C3). If it still misses, the miss is not unique to Flash. Never pool models. If the model lacks vision, drop screenshot (C1) and say so.
2. **Ablation.** If view document (C4) stays more expensive after flags are stripped, the overhead is document growth. If step savings disappear, the step win was constraint-carrying.
3. **Latency.** Fewer round trips could flip the verdict without changing token ranks.
4. **A harder application.** MiniShop is where flat tools (C3)'s tracking tax is smallest. "Flat tools (C3) wins on MiniShop" is not "do not ship view document (C4)."

We treat view document (C4)'s fewer steps as mechanism present, not yet decisive on cost.

8. How an application could ship view document (C4)

The measured view document (C4) is hand-authored, an upper bound. Almost all of it is validity logic the human UI already has: disable Add to cart until a size is chosen, validate the address, hide Pay until cart and address are set. Adoption is mostly *serializing existing predicates*, not inventing new rules. The backend stays the authority. The document may never advertise an action the backend would reject.

Five paths, from highest leverage to most general:

1. **Framework-emitted.** One declaration yields the human page and the view document. Low effort at scale, high fidelity, needs framework support that is not yet common.
2. **Projected from a spec.** GraphQL, form schemas, or server-driven UI already name types, mutations, and validation. Medium effort, fidelity high where the spec is the truth.
3. **Hand-authored (this study).** Faithful, does not scale. In MiniShop the view document (C4) projection is roughly three to four times the size of the flat tools (C3) definition, and it mirrors checks the human templates already encode.
4. **Compiled from the human page.** Read DOM, ARIA, and `disabled`. Near zero cost to the app, but lossy, and it can disagree with the backend. Must be validated.
5. **Model-extracted.** A model reads the screen and emits the document. Universal, and it puts the cost back on every step, with hallucination risk. A fallback, not the point of view document (C4).

Build time versus runtime. View document (C4) pays once (author, framework, or compile) and then states what is allowed. Flat tools (C3) pays a perpetual tax: the model re-derives state from

history on every step. That tax is small on MiniShop. It is expected to grow with more views, more state, and more ways to act illegally.

A short procedure for a developer today. List the views. For each view, expose state, entities, and actions with the same enable-conditions the human UI already uses. Serve the document. Keep backend enforcement. Add a consistency check so the two surfaces cannot drift.

Table 4. Adoption paths for view document (C4).

Path	App effort	Coverage	Fidelity	When
Framework-emitted	Low at scale	High	High	New apps; an agent-aware framework
Derived from a spec	Low to medium	Medium to high	High where the spec rules	GraphQL, form schemas, server-driven UI
Hand-authored	Medium per app	Whatever is authored	High	A few high-value flows; this paper's upper bound
Compiled from the human page	Near-zero	Broad	Lossy; must check	Legacy apps
Model-extracted	Near-zero	Universal	Lowest; recurs at runtime	Fallback only

Validating compiled and model-extracted surfaces against this hand-authored bound is future work.

9. Limitations

External validity. One model, one small synthetic shop, ten tasks. Paired intervals resample over 7 success tasks and 3 refusal tasks. We do not generalize beyond a controlled shopping task.

Construct validity of view document (C4). Hand-authored, so an upper bound. A badly written document would understate view document (C4). We freeze it and pin it with consistency tests.

Confounded mechanisms. Compactness and constraint-pruning are not separated until the ablation exists.

Refusal tasks can pass by inaction. They are reported separately.

Image tokens are provider-dependent. Screenshot (C1) cost is read from total input.

Determinism. Temperature zero is not full determinism. We average five repeats. Flat tools (C3) was stable; view document (C4) varies on t10; accessibility tree (C2) is noisy.

History. History is task, prior actions, and the current observation. That holds the policy fixed. It also means flat tools (C3) cannot accumulate a long transcript of past views. A different history policy is not tested.

screenshot (C1) illegal metric. Coordinate clicks rarely set the backend flag, so 0.000 illegal/step understates invalid clicks. For screenshot (C1), read success, steps, and tokens.

Power. A CI that includes zero is not proof of equality. We do not claim significance we do not have.

Models. The main line uses general models that can do all four conditions in one API. Specialized computer-use models are out of the main line. RQ3 has not been run.

10. Future work

Immediate: latency; a second general model; the constraint-pruning ablation; an ignored-affordance metric. Longer term: a second, harder application; a validated Agent Experience (AX) metric suite as a later paper; methods for generating the surface automatically, measured against the hand-authored bound here.

Appendix: how to reproduce the numbers

The shop, the four-condition harness, the grader, and the traces behind every number live in the project repository. There is no hidden state. In order:

1. A scripted, non-model policy must complete every success task on flat tools (C3) and view document (C4) and be refused on the refusal tasks.
2. The model-free observation-cost baseline (Section 6.1) uses a fixed tokenizer and no API call.
3. The oracle (Section 6.2) checks the pipeline on the two structured conditions.
4. The model sweep writes 200 traces (Section 6.3).
5. The report layer aggregates traces into Tables 1 to 3 and the bootstrap intervals.

Python dependencies and the browser used for screenshot (C1) and accessibility tree (C2) are pinned in the repository. This manuscript rebuilds to HTML and PDF from the Markdown source.

References

Verified sources only. Titles and URLs below were checked against the primary paper or official documentation. No additional citations are introduced here.

- WebArena: A Realistic Web Environment for Building Autonomous Agents.
<https://arxiv.org/abs/2307.13854>
- VisualWebArena: Evaluating Multimodal Agents on Realistic Visually Grounded Web Tasks. <https://aclanthology.org/2024.acl-long.50/>
- Mind2Web: Towards a Generalist Agent for the Web. <https://arxiv.org/abs/2306.06070>
- WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents.
https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf
- AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents.
<https://arxiv.org/abs/2405.14573>
- MiniWoB++ (Reinforcement Learning on Web Interfaces using Workflow-Guided Exploration). <https://arxiv.org/abs/1802.08802> ; docs: <https://miniwob.farama.org/>
- Anthropic: Introducing computer use (Claude 3.5 Sonnet).
<https://www.anthropic.com/news/3-5-models-and-computer-use>
- UI-TARS: Pioneering Automated GUI Interaction with Native Agents.
<https://arxiv.org/abs/2501.12326>
- SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents.
<https://aclanthology.org/2024.acl-long.505/>
- UIFormer / From User Interface to Agent Interface: Efficiency Optimization of UI Representations for LLM Agents. <https://arxiv.org/abs/2512.13438>
- Prune4Web: DOM Tree Pruning Programming for Web Agent.
<https://ojs.aaai.org/index.php/AAAI/article/download/40772/44733>
- A2UI (Agent-to-UI) Protocol. <https://a2ui.org/>
- Model Context Protocol (MCP). <https://modelcontextprotocol.io/>